

TradeBot — Indian Equity Screening Engine

Project guide: how it works, what is done, what is left

Repository: github.com/Amogh2409/Stock_Engine · Branch `main` @ `db97f7f` · 31 commits
Generated 13 September 2026 · All figures verified against the repository and its measurement artefacts

Read this before anything else. The technical scoring model has now been measured against eleven years of real prices, and **it does not predict returns**. Rank correlation with next-month returns is -0.0015 ($t = -0.09$, $n=130$) where the smallest detectable effect is 0.046 — a properly powered null, not a shortage of data. All four scoring blocks are independently flat. Ranking by this score returned 20.40% a year against 21.15% for simply equal-weighting the same 100 stocks.

The engine is well-built, well-tested and honestly documented. What it is *not*, on current evidence, is a stock picker. Everything in this guide should be read with that in view.

1. What this project is

A deterministic screening engine for Indian equities. It reads a Screener.in CSV export of company fundamentals, optionally reads a price-history CSV, scores every company out of 100, ranks them, and produces a watchlist with position sizes and stop levels — in the browser, in Python, and in a Colab notebook, all producing identical numbers.

The stated goal versus where it actually is

The goal	Current reality
Continuous Nifty 100 analysis	Runs on demand from a manual CSV upload. No automatic daily run, because there is no free fundamentals feed.
Buy / sell guidance	Produces entries only . Position sizing and stops now exist; there is still no sell rule or exit discipline beyond the next month's re-rank.
Percentage allocations	Done. Equal-risk sizing by volatility, with an explicit cash position.
Learn from the trading books	Done as documentation, not as automation. Every scoring rule is traced to a source or explicitly marked unsourced in <code>knowledge/rulebook.md</code> .

Research only. The engine never places an order, never connects to a broker, and has no live trading path. Everything it produces is a research signal for a human to act on or ignore.

2. Quick start

First-time setup

```
cd ~/PersonalResearch/TradeBot

# Python environment (needed by the engine and all Python tests)
python3 -m venv .venv
.venv/bin/pip install -r requirements-test.txt      # offline runs + tests
.venv/bin/pip install -r requirements-network.txt  # adds live price download

# Node dependencies
npm ci
```

The five commands that matter

Command	What it does
<code>npm run dev</code>	Starts the browser app on localhost. Upload a CSV, see the watchlist.
<code>npm run screen:offline</code>	Runs the Python engine with no network, using files in <code>data-store/</code> .
<code>npm run verify</code>	The full release gate — 24 steps, clean-room build, ~20 min. Run before pushing.
<code>npm test</code>	Frontend and cross-engine parity suite (234 tests).
<code>.venv/bin/python python/backtest.py --prices <csv> --out-dir <dir></code>	Re-runs the measurement. ~90 seconds against the cached price file. Stops at 2023-01-01 by default — see the reserved window below.

A reserved window now exists, and it changes what a re-run returns. 2023-01-01 onward is held back (`knowledge/holdout.md`). The backtest truncates there by default, which surrenders **45 of 141 rebalances — 32% of the sample** — and leaves roughly eight independent 12-month observations.

The figures in section 5 of this guide were measured across the *full* span before that cut existed. To reproduce them exactly, pass `--no-respect-holdout`. Do not make a habit of it: that flag exists for one final test, and every casual use spends the window permanently.

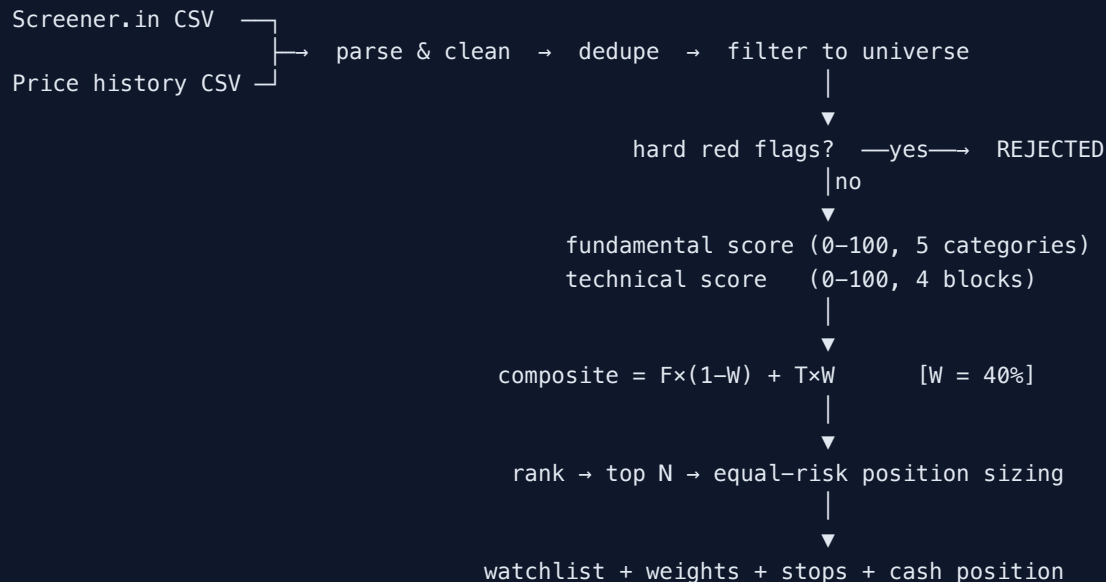
The four checks before any commit

```
npm run lint          # tsc --noEmit          → expect exit 0
npm run check:python  # generated files in sync → expect exit 0
.venv/bin/python test_python.py      # 161 tests → expect exit 0
npx vitest run        # 234 tests → expect exit 0
```

Read each exit code from the command that produced it. `cmd | tail` returns *tail's* exit code, not the command's — this has already caused one false "passed" report in this project.

3. How it works

The pipeline, end to end



Two engines, one answer

The same logic exists twice — `python/engine.py` (5,500 lines) and `src/utils/screenerEngine.ts` (3,809 lines). They are not approximately equal; they are asserted **bit-identical** by 39 parity tests that run both engines on the same data and compare every score, every text string and every CSV byte.

This is why rounding is `round1()` everywhere (half-up, one decimal) and why sorting uses code-point order rather than locale order. A rounding difference is a ranking difference.

Generated files — do not edit by hand. `src/utils/pythonEngineSource.ts` and the Colab notebook are both built from `python/engine.py` by `npm run build:python` and `npm run generate:notebook`. Editing the engine without regenerating them makes `check:python` fail, which is exactly what it is for.

Where data lives

Path	Contents
<code>data-store/fundamentals/</code>	Screener.in exports. Currently only the invented sample.
<code>data-store/fundamentals/archive/</code>	Dated, hashed copies of every export ever used — the point-in-time history.
<code>data-store/market_data/</code>	Downloaded price history (26 MB, 266,432 rows, 2015–2026).
<code>data-store/reports/</code>	Watchlist CSVs, rejection audits, backtest artefacts.
<code>knowledge/rulebook.md</code>	961 lines. Every scoring rule with its source, or an admission that it has none.

`data-store/**` is gitignored apart from the directory skeleton, so measurement artefacts do not travel with the repository. Anything that must survive belongs in the rulebook.

4. The scoring model

Step 1 — hard red flags (rejection, not a low score)

These mean "this row cannot be trusted as evidence", not "this company is unattractive". An expensive stock still gets scored; a company with nine of these does not.

- **Negative net worth** — negative P/B or D/E is insolvency, not a bargain
- **High promoter pledge** — above the configured limit
- **Negative or missing operating cash flow** — non-financial companies only; a growing loan book legitimately consumes cash
- **Insufficient data** — too little of the row present to score honestly
- **Non-positive sales, market cap, or impossible shareholding percentages** — the row is corrupt
- **Pledge without promoter holding** — internally contradictory
- **Conflicting duplicate rows** — two rows disagree, so neither can be believed

Step 2 — fundamental score, 100 points

Category	Points	Measures
Financial quality	30	ROCE, ROE, margins
Growth	25	Sales and profit growth
Balance-sheet safety	20	Debt/equity, interest cover, cash flow
Valuation	15	P/E and P/B <i>against the sector median</i> , not absolute
Governance	10	Promoter holding and pledge

Banks and NBFCs are scored by a separate model using return on assets, NPAs, capital adequacy and CASA — because leverage and interest cover do not describe a lender. A financial company whose export lacks those columns is reported as *"not scored: missing bank metrics"* rather than being scored wrongly on ratios that do not apply.

Step 3 — technical score, 100 points in four blocks

Block	Max	Rules
Trend	40	Price > 200 SMA (12), > 50 SMA (8), 50>200 cross (8), ADX>25 (4), within 15% of 52-week high (8)
Momentum	30	MACD histogram rising (15), RSI > 50 (15)
Relative strength	20	Beat the benchmark over 3m (5), 6m (10), 12m (5)
Volume	10	Above 20-day average (5), positive OBV pressure (5)

Volatility, overbought RSI and deep drawdowns are **flags, never points**. Paying for low volatility tilts toward sleepy stocks and paying for high volatility toward lottery tickets; neither is defensible, and the source treats volatility as a risk measure rather than a directional edge.

A company needs **200 sessions** of price history to be scored at all. Below that the score is *null*, not a low number — a short listing is not a weak company.

Step 4 — the composite, and what it gates

$$\text{composite} = \text{fundamental} \times (1 - W/100) + \text{technical} \times (W/100) \quad W = 40$$

The pass mark (`minimum_total_score = 50`) gates the **fundamental** total. The composite decides *ordering*, not admission. Companies that pass but cannot be priced are ranked in a separate list, because a composite without a technical half is not on the same scale — mixing them would reward being absent from the price file.

Step 5 — position sizing

Equal risk by volatility (weight proportional to $1/\text{volatility}$, normalised), scaled to a 12% portfolio volatility target, then capped per position and per sector. Stops at 2× ATR. The shortfall stays in cash and the app says why.

This is the best-sourced part of the system — real page citations, a worked table from the source, and an explicit decision rule. Note the direction of that argument: equal risk is what the book prescribes *precisely because* the selection underneath it is unproven. The sizing citations lend the scoring layer nothing.

5. What has been measured

Backtest: 266,432 price rows, 101 tickers, 2015-01-01 to 2026-09-11, monthly rebalance, next-day-open execution, 15 bps per side. No look-ahead by construction.

These figures span the full period, measured before the reserved window was introduced. The backtest now stops at 2023-01-01 by default, so a plain re-run returns numbers over 2015–2022 instead and will not match the tables below. Add `--no-respect-holdout` to reproduce them. The reserved window is documented in `knowledge/holdout.md`, which states honestly that it is *not* pristine: it was observed once, on 13 September 2026, at composite and block level. What has not happened is the thing that actually matters — no engine parameter was changed in response to it. Used once for observation, never for fitting.

The headline null

Horizon	Rank IC	t	n	p (corrected)	Detectable
1 month	−0.0015	−0.09	130	1.000	0.046
3 months	+0.0481	+2.46	43	0.358	0.056
6 months	+0.0278	+0.82	21	1.000	0.100
12 months	+0.0843	+2.05	10	1.000	0.128

Why the 1-month row carries the whole argument. A 1-month forward return sampled monthly *cannot overlap*, so the two possible estimates coincide exactly. Every longer horizon must either reuse $(h-1)/h$ of each window — inflating the t-statistic — or keep one phase offset in h and collapse the sample. At 12 months the independent sample is **$n = 10$** and cannot detect an IC below 0.128, against a realistic effect of 0.02–0.05. A null there means the test could not see, not that nothing is there.

No block carries it

A near-zero total is equally consistent with four dead blocks and two that cancel out. Ranking on each subtotal alone settles it. At 1 month — the only horizon with real power — every block sits inside its own detection floor:

Block	IC	t	Detectable at 80% power
Trend (40 pts)	+0.0035	+0.21	0.048
Momentum (30 pts)	−0.0126	−0.96	0.037
Relative strength (20 pts)	+0.0140	+0.88	0.045
Volume (10 pts)	−0.0126	−1.07	0.033

Four flat blocks. Nothing is cancelling out, so no reweighting recovers an edge that is not there.

Portfolio results

Window	Top 20 by score	Equal-weight all 100	Nifty index
Full period	20.40%	21.15%	9.07%
In sample (to 2021)	17.28%	20.18%	10.39%
Out of sample (2022+)	24.78%	22.63%	7.09%

Beat equal-weighting in **4 of 12** calendar years, and **1 of 7** in-sample — losing 2021 by 19.5 points and 2017 by 14.6. Annual turnover **754%**.

Three numbers that look like findings and are not

1. The out-of-sample 24.78% vs 22.63%. That window was already used to revise the engine. Once out-of-sample data has been used to fix something, what comes back is overfitting — it is an in-sample number wearing an out-of-sample label. At 754% turnover it is negative after Indian short-term capital gains tax in any case.

2. The 3-month $t = +2.46$. Its sign *reverses* against the overlapping estimate (-0.11). A sign that flips with the sampling phase is an artefact, not a signal.

3. The RSI flip beating equal-weight in all three windows (22.88 / 20.46 / 26.26). Tested both directions; the rank IC underneath is $+0.0059$. With essentially zero IC the top-20 composition is noise, so what changed is *which twenty names the noise picked* — a story about twenty draws, not about a rule. Neither sign is adopted.

6. Status — what is done

28 commits, all independently verified in a clean room by a second session before pushing.

DONE Core engine and parity

- Two-engine architecture with 39 byte-identical parity tests
- Financial-company model (banks/NBFCs scored on their own metrics, or honestly not scored)
- Sector-relative valuation with a minimum sample size and a stated fallback chain
- Hard red flags separated from low scores
- Unpriced companies ranked separately rather than silently dropped

DONE Technical analysis

- 18 chart indicators: RSI, MACD, ADX/DI, ATR, Bollinger %B, OBV, ROC, drawdown, relative strength
- Wilder smoothing, EMA seeding, population sigma — matching the textbook formulas exactly
- Four-block scoring with per-block subtotals exposed
- Sector-relative strength as a diagnostic (earns no points, computed after scoring so it structurally cannot reach one)
- Price charts in the detail panel, drawn against the levels the score actually used

DONE Position sizing and risk

- Equal-risk sizing by volatility, scaled to a 12% portfolio target
- Per-position and per-sector caps; 5% per-trade risk ceiling
- Stops at 2× ATR, set in advance
- Cash position shown and explained in the app, the CSV and the HTML report

DONE Measurement and honesty infrastructure

- Backtest with no look-ahead, next-day-open fills, modelled costs
- Both overlapping and non-overlapping sampling reported side by side, with sign-flip flagging
- Every IC printed with its sample size *and* its minimum detectable effect
- Bonferroni correction across the whole family of 20 tests
- Student's t rather than a normal approximation (at df=9 the difference decides significance)
- The backtest's own mechanics are now inside the release gate, and proven to fail when broken
- A reserved window from 2023-01-01, enforced in the one function every caller passes through, so a new caller inherits the protection instead of having to remember it
- Newey-West standard errors for overlapping windows, validated against series with known overlap structure and cross-checked by an independent bootstrap
- `knowledge/rulebook.md` — 961 lines tracing every rule to a source or marking it unsourced
- Dated, hashed archive of every fundamentals export used

7. Status — what is left

BLOCKED ON YOU A real Screener.in export

This is the single largest blocker in the project. `data-store/fundamentals/` contains only `sample_screener_export.csv`, whose figures are **invented**. The archive has run exactly once — on that sample (verified: identical SHA-256, and none of the four bank columns present).

What to do: in Screener.in, add these four columns to the screen *before* exporting, then drop the CSV into `data-store/fundamentals/` :

- Return on assets
- Gross NPA %
- Net NPA %
- Capital adequacy ratio

What it unblocks:

- ~23 financial companies get scored for the first time instead of "missing bank metrics"
- The provisional pass mark of 50 gets real data to calibrate against
- Point-in-time history starts accumulating — the *only* route by which the fundamental half ever becomes testable

When it arrives, split it before looking at it. Calibrate on part, and stamp the rest unexamined. Tuning the pass mark against a snapshot you have already studied spends the only clean data available and leaves the threshold exactly as unfounded as it is today, with better-looking provenance.

YOUR DECISION The technical weight

`technical_weight_pct = 40` is now **unearned**, not merely unvalidated. Measured at six real rebalance dates, the technical score spans the full 0–100 range every month (mean span 99.3). So at $W=40$ the technical half can swing a composite by **39.7 points** — enough to move a company across all three verdict bands (Strong ≥ 70 , Good ≥ 60 , Average ≥ 45) on its own. It is frequently the deciding term, and it is deciding on a measured null.

Option	Meaning	The honest cost
40 current	Status quo	~40 points of ordering driven by a measured null
10–20	Reduce the null's influence	No evidence picks any value over another — arbitrary, but smaller
0	Rank on fundamentals alone	Not the safe option. Concentrates 100% of ordering on the half that is <i>untested</i> — trades a known null for an unknown
0 + display	Technicals shown, never ranked	Keeps charts and flags as information without letting them order anything

Not computable today: how the actual watchlist *composition* shifts as W varies needs both halves, and the fundamentals half would be invented numbers. That calculation waits on the real export.

NOT STARTED Exits and sell discipline

The engine produces **entries only**. There is no exit rule, no trailing stop logic, and no sell signal beyond next month's re-rank. Against the stated goal of buy/sell guidance, this gap is structurally larger than the pass mark or any remaining scoring work. Knowing your exit in advance is basic risk management in the source material.

Deliberately unclaimed: adding exits turns a screener into a trading system, which is a product decision rather than an engineering one.

OPEN Rulebook items, ranked

#	Item	Why it matters
1	Hold data back, permanently	Fix a window now, never look at it. Until then no out-of-sample claim survives. Blocked on the export.
2	Map robustness, not points	Sweep thresholds looking for a plateau rather than a spike. An erratic surface also catches harness bugs.
3	Add price spread	The missing input behind both the buying-climax conflict and the incomplete OBV implementation.
4	Replace $ATR\% > 5$ with relative volatility	Swaps an invented constant for a sourced construction, $ATR(14)/ATR(140)$.
5	Reconsider ADX as a gate	Trend rules measure nothing in a sideways market; 4 points beside them is the wrong shape of rule.
6	Decide what the relative-strength points are for	20 points on convention alone; its one citation argues <i>against</i> the two-point form used.

8. Known weaknesses — read before trusting any number

Structural limits of the measurement

- **Survivorship bias.** Today's Nifty 100 applied to history. Companies that fell out of the index are absent and are disproportionately the bad ones. Every figure, including the benchmarks, is flattered.
- **The fundamental half has never been tested.** Not "tested and found weak" — never measured at all. Today's export describes today, so applying it to 2016 would be look-ahead of the worst kind.
- **No dividends, no taxes.** Costs are modelled; Indian STCG is not, and at a monthly rebalance it is the larger drag.
- **The books teach single-instrument timing, not cross-sectional ranking.** An indicator can time one stock well and carry no information about which of a hundred will outperform the other ninety-nine.

Rules that are convention, not evidence

Roughly half the live technical thresholds are inherited market convention with no source in any book in the repository: the 50/200 SMA pair, RSI 50, ATR 5%, the 2× ATR stop multiple, the 15% 52-week-high proximity, and the 5/10/5 relative-strength split. They are not wrong — they are *unjustified*, and the rulebook labels each so nobody later assumes a number was measured when it was inherited.

Three places the books actively disagree with the engine

- **RSI direction.** Scoring RSI > 50 treats it as continuation. The source's own evidence shows the sign *flipped* around 1998 — a trend indicator before, mean-reverting since. The risk is not that the rule is unproven; it is that it may be directionally wrong in some regimes.
- **High volume near the highs.** The engine pays 13 points for a configuration one source calls a *buying climax* and reads as bearish. The distinguishing feature is price spread, which the engine does not compute.
- **Stops may not help at all.** The source is sceptical that stop-losses improve trend or mean-reversion profiles, and gives multiples of 3× and 12× ATR — neither of which is the 2× used here. The UI says so rather than letting a stop price imply a protection the books endorse.

9. Working rules — learned the hard way

Every rule below came from an actual failure in this project. They are worth following because each one has already cost time.

Before trusting a green signal, ask what it would look like if the thing were broken

If the answer is "the same", the signal carries no information. Four real instances in one day:

- A look-ahead test compared a score to itself at fixture session 66 when scoring needs 200 — both sides were null, so `assertEqual(None, None)` passed for as long as the test existed, while the docstring claimed the test asserted no-look-ahead.
- A guard returning bare `None` crashed on the first real run, because every test fixture gave every ticker every date. **A guard for absent data is only exercised by absent data.**
- A flag was "proved" to work by printing `DIFFERENT: True` from `nan != nan`, which is true for any two NaNs.
- A failing Python run reported exit 0, because the code came from `tail` at the end of a pipe rather than from Python.

A grep count is not evidence until you have read the lines

Both halves bite. A zero count means nothing unless the pattern is shown to match text known to exist — four false zeroes here came from shell glob mangling. And a non-zero count says nothing about *what* matched: four occurrences of `__dirname` were assumed to be four comments; three were, and the fourth was live code that shipped under a commit message claiming it was gone.

Verify against the whole of a thing, not its first screen

Three rulebook claims were built on the seven files at the top level of the books folder while 128 more sat in a subdirectory — which made two of them false. Separately, "~71 entries" was reported from a directory *link count* rather than a file count. Neither was an invented number: each was an accurate figure attached to a claim it did not support.

A test can only exercise the range its fixture spans

Portfolio volatility was measured over the whole price file — an eleven-year mean — which left the crisis brake inert, because an eleven-year average cannot rise. Every unit test passed, both engines agreed to the last bit, and every fixture was shorter than the window that was wrong. It was caught by reading a real run's output and noticing a number that was impossible.

Once out-of-sample data has been used to fix something, it is spent

This is the rule the project has broken once and documented rather than hidden. The sequence was: measure, find a null, revise the engine in response, re-measure *on the same window*. The resulting +2.15% cannot be cited. Only data nobody has examined settles whether a change helped.

Repository hygiene

- Stage explicit paths. Never `git add -A` — it once swept 77 lines of someone else's half-written work into an unrelated commit, cleanly and with exit code 0.
- Never run the generators (`build:python` , `generate:notebook`) while someone else owns `engine.py` . They read like checks and write like builds.
- Read `git diff --cached --stat` before every commit and abort on an unexpected file.

10. File map

Path	Lines	Role
python/engine.py	5,500	The engine. Source of truth; the notebook and the TS mirror are generated from it.
src/utils/screenerEngine.ts	3,809	Browser engine. Bit-identical to the Python by contract.
python/backtest.py	1,198	The measurement. No look-ahead, next-day-open fills, both samplings.
knowledge/rulebook.md	961	Read this first. Every rule, its source, and what it does not rest on.
knowledge/holdout.md	—	What is reserved, why, and the honest admission that it is not pristine.
README.md	553	Operational detail and data contracts.
src/types.ts	397	Shared types, mirroring the Python dicts field for field.
python/ab_compare.py	166	A/B harness for comparing two engine revisions on identical data.
run_checks.sh	461	The 24-step release gate: clean-room build, all suites, notebook byte-comparison.
src/components/	11 files	UI. <code>WatchlistTable</code> , <code>PositionSizingSummary</code> , <code>PriceChart</code> , config, audit views.
src/tests/	8 files	234 frontend/parity tests; <code>test_python.py</code> adds 161.

Test inventory

Suite	Count	Covers
vitest (frontend + parity)	234	Engine logic, UI rendering, cross-engine agreement
test_python.py	161	Python engine contracts, notebook cells
backtest --self-test	10	No look-ahead, execution timing, costs, p-values, Bonferroni
run_checks.sh	24 steps	All of the above in a clean room, twice, plus archive re-verification

11. If you are picking this up cold

1. **Read** `knowledge/rulebook.md`. It is the honest account of what every number rests on. Its "Open items" section is the real backlog.
2. **Run** `npm run dev` and upload the bundled sample to see the output shape. Remember the sample's figures are invented.
3. **Run the backtest** against the cached price file (~90 s) to reproduce the null yourself rather than trusting this document. It stops at 2023-01-01 by default, so the numbers will be close to but not identical with section 5; add `--no-respect-holdout` only if you need an exact match, and understand what that flag costs before you use it.

4. **Do not tune thresholds to improve the IC.** That is the trap this repository has already documented against itself. Measuring is fine; adopting on data already used is fitting.
5. **Leave the reserved window alone.** 2023 onward is held back for one final test. `--no-respect-holdout` exists for that test, not for curiosity.
6. **Get the real Screener export in.** Almost everything else is downstream of it.

The honest summary. This is a carefully built, thoroughly tested, unusually well-documented engine that has been measured and found not to predict returns. That is a real and useful result: most such projects never find out. The infrastructure to test the *fundamental* half — which is 60% of the composite and has never been measured — is built and waiting on one CSV file.

Generated 13 September 2026 from the repository at commit 163ae40. Every statistic here was read from `data-store/reports/backtest_blocks/results.json` and `backtest_rsiflip/results.json` programmatically rather than transcribed by eye. Research and paper-trading only — the engine places no orders.